

Clipper — Gerador de Cortes

Documentação técnica completa do projeto (referência)

Descrição curta: transcreve lives/gameplay localmente, pontua os melhores momentos com LLM e entrega clipes verticais 9:16 legendados.

Data da documentação: 2026-09-18 **Commit:** c0aa055 (pós-auditoria) **Status:** funcional, aprovado para teste em vídeo real (SAFE TO BEGIN VIDEO TEST)

Como usar este documento: referência completa e autocontida — instalação, configuração, CLI, arquitetura, GPU, API, métricas, troubleshooting e auditoria de estabilidade.

2. Sumário

2. Sumário	2
3. Visão geral	4
4. Quick Start	4
5. Requisitos	4
5.1 Hardware	4
5.2 Sistema operacional	5
5.3 Software	5
5.4 Dependências Python	5
6. Instalação	5
7. Configuração	6
7.1 Variáveis de ambiente	6
7.2 Constantes internas (core/config.py)	6
7.3 Diretórios configuráveis	7
8. CLI completa	7
VIDEO (posicional, obrigatório)	7
--out PATH (padrão: cortes)	7
--top N int (padrão: 8)	7
--model NOME (padrão: env NIM_MODEL)	7
--transcribe-backend {auto,gpu,vulkan,openvino,cpu} (padrão: env ou auto)	7
--pad SEG float 0–5 (padrão: 0.8)	7
--min-score F 0–10 (padrão: 6.0) + --max-per-10min N 1–20 (padrão: 2)	7
--no-vertical / --no-captions / --no-audio-features (booleanas)	7
--cache-dir DIR + --force-retranscribe + --force-rescore	8
--context TEXTO + --examples JSON	8
9. Exemplos de execução	8
10. Fluxo completo do sistema	8
11. Arquitetura	9
12. Estrutura de diretórios	9
13. Transcrição	10
14. GPU e aceleração	10
15. Processamento de vídeo	11
16. NVIDIA API	11
17. Métricas	11
17.1 Referência de campos	12

18. Cache	13
19. Saídas	13
20. Recursos e limites	13
21. Segurança e estabilidade	13
22. Tratamento de erros	14
23. Troubleshooting	14
GPU não detectada	14
OpenVINO falhou	14
Vulkan falhou	14
Caiu para CPU — por quê?	14
FFmpeg falhou	14
NVIDIA API falhou	15
Falta de RAM	15
Falta de VRAM	15
Falta de disco	15
Processo interrompido	15
24. Benchmark	15
25. Desenvolvimento	15
26. Testes	16
27. Dependências	16
28. Limitações conhecidas	16
29. Auditoria de estabilidade	17
HIGH corrigidos	17
MEDIUM / LOW (resumo)	17
30. Changelog / estado atual	17
31. Referência rápida	18

3. Visão geral

O que é: programa Python de linha de comando que recebe a gravação de uma live ou gameplay (mp4, mkv etc.), transcreve o áudio **localmente e offline**, identifica os melhores momentos com um LLM (tier grátis da NVIDIA Build) e gera os clipes cortados em **9:16 com legenda queimada**, prontos para TikTok/Reels/Shorts, mais *manifest.json* e um **relatório de métricas por execução**.

Problema que resolve: garimpar manualmente horas de live para achar 8 cortes virais. Fluxo pensado no ritmo diário: sobe o vídeo de manhã, os clipes ficam prontos ao longo do dia, custo zero.

Funcionamento em alto nível: validação fail-fast → transcrição local (GPU Intel primeiro, CPU como fallback explícito) → janelas candidatas 20–90 s → energia de áudio → scoring em lotes via API → seleção com diversidade → corte/legenda/encode com ffmpeg (Quick Sync na B580) → relatório JSON.

Principais tecnologias: Python 3, faster-whisper/CTranslate2 (CPU), OpenVINO ou whisper.cpp/Vulkan (GPU Intel), ffmpeg/ffprobe, OpenCV Haar (rosto), OpenAI SDK contra a API NVIDIA, psutil (métricas).

```
python clipper.py video-1h40.mkv --out cortes/ --top 8
# ■ cortes/01_*.mp4 ... 08_*.mp4 + manifest.json
# ■ metrics/2026-09-18_112045_video-1h40_a1b2c3.json
```

4. Quick Start

Do zero até o primeiro corte (Fedora, máquina com B580 — comandos reais, validados):

```
sudo dnf install -y ffmpeg intel-level-zero intel-media-driver
pip install -r requirements.txt
# (opcional, p/ transcrever na B580) pip install openvino optimum-intel transformers soundfile
export NVIDIA_API_KEY="nvapi-sua-chave"; # https://build.nvidia.com (grátis)
python clipper.py --help # smoke test: abre a CLI
python clipper.py minha_live.mp4 --out cortes/ --top 8
```

O que cada passo faz: instala sistema+drivers; instala Python; registra a chave do scoring (sem ela o preflight aborta); valida a CLI; processa o vídeo gerando clipes + *manifest.json* + relatório em *metrics/*. Detalhes por camada na §6; todas as flags na §8.

5. Requisitos

5.1 Hardware

Item	Mínimo prático	Recomendado / testado
CPU	4 threads livres (código exige ≥ ~6 lógicos p/ limites são)	Ryzen 5 4600G 6C/12T (medido)
RAM	8 GB (código usa modelo small se ≤ 6 GB)	14–16 GB (pico medido ~0,9 GB num clipe de 55 s; estimado poucos GB p/ 1h40)
GPU	Nenhuma (CPU funciona, mais lento)	Intel Arc B580 12 GB (encode QSV verificado; transcrição GPU exige driver)
VRAM	—	Whisper-medium em GPU ≈ 2–4 GB de 12 GB; margem folgada
Disco	Vídeo + 1 GB livre (preflight falha abaixo disso)	707 GB livres medidos; saídas ≈ dezenas de MB + cache de MBs

5.2 Sistema operacional

Suportado/testado: **Linux — Fedora 44 (Xfce)**. Na prática o código é Linux-dependente em pontos reais: detecção GPU via `/sys/class/drm` e `lspci`, RAM via `/proc/meminfo` (fallback), CPU via `/proc/cpuinfo`. macOS/Windows não são testados (README cita brew apenas para ffmpeg).

5.3 Software

Software	Versão testada	Para quê
Python	3.14.7 (código usa sintaxe 3.10+)	runtime
ffmpeg + ffprobe	8.1.2	chunks, volumedetect, corte, encode, sondas
intel-level-zero, intel-media-driver	ausentes aqui → CPU explícito (verificado)	expor a B580 ao OpenVINO/ffmpeg
whisper.cpp (binário Vulkan) + modelo ggml-medium	ausente aqui → não usado	transcrição na B580 (alternativa)
vainfo / lspci	VA-API 1.23	diagnóstico (não obrigatórios)

5.4 Dependências Python

Dependência	Versão	Finalidade	Obrigatória
whisper	1.2.1	transcrição CPU (Whisper medium int8)	Sim
	3.15.0	client HTTP do scoring (contra API NVIDIA)	Sim
python	5.0.0.93	detecção de rosto (3 frames/clipe)	Sim
	2026.4.0	devices + inferência Whisper na B580	Só Linux; só p/ backend openvino
	7.2.2	métricas CPU/RAM (importada em metrics.py/system.py)	Sim (requirement)
intel, transformers, soundfile	—	carregar Whisper no OpenVINO + ler WAV	Só p/ backend openvino
	5.0.1	gerar PDFs de documentação (não é runtime)	Não

6. Instalação

```
sistema (Fedora)
sudo dnf install -y ffmpeg intel-level-zero intel-media-driver
vainfo | grep -i intel          # confirma driver de vídeo
↓ dependências Python
pip install -r requirements.txt  # faster-whisper, openai, opencv, psutil
pip install openvino optimum-intel transformers soundfile  # só se for transcrever na B580
↓ GPU/backend
python3 -c "import openvino as ov; print(ov.Core().available_devices)" #
esperado: [CPU, GPU]
# alternativa: binário whisper-cpp/whisper-cli no PATH + models/ggml-medium.bin
↓ configuração
export NVIDIA_API_KEY="nvapi-..."; export NIM_MODEL="z-ai/glm-5.3"
↓ teste (sem vídeo real)
python clipper.py --help
python clipper.py /tmp/inexistente.mp4  # deve falhar no preflight COM relatório em
metrics/
```

Caminhos separados: (A) CPU-only — só ffmpeg + requirements + API key; (B) +QSV — drivers Intel (encode GPU verificado por sonda); (C) +transcrição GPU — (B) mais stack OpenVINO ou whisper.cpp. Cada camada é opcional e o programa informa o que está ativo (nunca silencioso).

7. Configuração

Não há arquivos de configuração (YAGNI) — tudo é variável de ambiente + flags CLI. Constantes internas (janelas, legendas, timeouts) vivem em `core/config.py` e exigem edição de código.

7.1 Variáveis de ambiente

Nome	Tipo	Padrão	Obrigatório	Descrição
NVIDIA_API_KEY	string	—	Sim (scoring)	Chave grátis de build.nvidia.com; sem ela o preflight aborta
NIM_MODEL	string	z-ai/glm-5.3 (decisão original; vivo em 2026-09-18)	Não	Slug do modelo de scoring (--model sobrepõe)
CLIPPER_TRANSCRIBE_BACKEND	auto gpu vulkan openvino cpu	auto	Não	Backend de transcrição (--transcribe-backend sobrepõe; gpu exige GPU)
WHISPER_MODEL_SIZE	tiny base small medium large ...	medium (small se RAM ≤ 6 GB)	Não	Tamanho do Whisper (todos os backends)
CLIPPER_LANGUAGE	string null	null (= auto)	Não	Idioma forçado da transcrição
CLIPPER_CPU_THREADS	int	4 (auto p/ 4600G)	Não	Threads do Whisper CPU + OMP/MKL/OpenBLAS
CLIPPER_FFMPEG_THREADS	int	2 (auto)	Não	Threads por processo ffmpeg
CLIPPER_COMPUTE_TYPE	string	int8	Não	Precisão CTranslate2 (CPU força int8 se inválido)
CLIPPER_DEVICE	string	cpu	Não	Device do faster-whisper (cuda nunca é assumido)
CLIPPER_RAM_LIMIT_GB	int	6 (auto)	Não	Desde a auditoria: informativo apenas (nenhum limite é imposto)

7.2 Constantes internas (core/config.py)

Constantes (valor)	Efeito
MIN 20 s, MAX 90 s, passo 20 s, 6/ lote scoring, overlap 0.25	Nº de candidatos e chamadas API
tol 1.5 s, pad 0.8 s, 32 chars/linha, 2 linhas, mín 1.0 s, pausa 0.4 s	Corte em frase + SRT
min-score 6.0, max/10 min 2, NMS 0.8	Filtragem e diversidade
API 180 s; ffmpeg 600/300/120/8 s; sondas 2–15 s	Nenhum subprocesso/rede sem teto
API 3× (10/30/60 s); GPU 1× cada; QSV→x264 1×	Sem retry infinito em nenhum ponto

7.3 Diretórios configuráveis

--out (padrão *cortes*), --cache-dir (desligado por padrão), --examples (desligado), metrics/ fixo (relatórios).
Modelos: ggml em *models/ggml-<tamanho>.bin* ou *~/.cache/whisper/* (whisper.cpp); HF hub
~/.cache/huggingface (faster-whisper/OpenVINO).

8. CLI completa

Uso: *python clipper.py VIDEO [--flags]*. Sem subcomandos nem aliases. Validações que abortam (exit, com relatório): --pad fora de 0–5, --min-score fora de 0–10, --max-per-10min fora de 1–20, --top ≤ 0 (preflight). Comportamento sem flags: top 8, 9:16 com legenda, backend auto, sem cache, modelo do env.

VIDEO (posicional, obrigatório)

Tipo: caminho (str, ~ aceito). Finalidade: vídeo de entrada (mp4, mkv etc.). Exemplo: *python clipper.py minha_live.mp4*. Interação: nunca é alterado; guarda impede saída com mesmo caminho resolvido.

--out PATH (padrão: cortes)

Diretório das saídas (criado se preciso; testado quanto a escrita). Exemplo: *--out cortes/*. Com --cache-dir, reuso permite --out diferente sem recusto.

--top N int (padrão: 8)

Quantos cliques gerar (após seleção). Deve ser > 0. Exemplo: *--top 5* com --cache-dir para re-derivar sem retranscrever.

--model NOME (padrão: env NIM_MODEL)

Slug do modelo NVIDIA p/ scoring. Valores: qualquer id vivo de */v1/models* (padrão atual verificado vivo em 2026-09-18).

--transcribe-backend {auto,gpu,vulkan,openvino,cpu} (padrão: env ou auto)

auto = whisper.cpp → OpenVINO/GPU → CPU com motivo impresso. gpu = exige GPU (Vulkan ou OpenVINO/GPU) e falha claramente sem fallback. Valores explícitos indisponíveis caem p/ CPU com motivo (nunca silencioso). Ex.: *CLIPPER_TRANSCRIBE_BACKEND=cpu python clipper.py v.mkv*.

--pad SEG float 0–5 (padrão: 0.8)

Respiro antes/depois do snap de frase. Ex.: *--pad 1.2* se cliques cortam no meio da frase.

--min-score F 0–10 (padrão: 6.0) + --max-per-10min N 1–20 (padrão: 2)

Filtragem e diversidade temporal da seleção. Ex.: *--min-score 5.0 --max-per-10min 1* p/ corte mais solto e espalhado.

--no-vertical / --no-captions / --no-audio-features (booleanas)

Mantêm widescreen (1280px) / desligam legenda / pulam volumedetect (scoring mais rápido). Ex.:
--no-audio-features acelera sem ffmpeg por candidato.

--cache-dir DIR + --force-retranscribe + --force-rescore

Ativa cache (transcrição + scores por fingerprint tamanho+mtime+modelo+idioma). Forçadores ignoram cada camada. Ex.: --cache-dir .cache/clipper.

--context TEXTO + --examples JSON

Injetam contexto no prompt e few-shot (lista ou {examples:[...]}, máx 4, exige transcript). Ex.: --context "ranked Valorant" --examples examples.json.

Combinações inválidas: valores fora das faixas acima; --transcribe-backend fora das 4 opções (argparse rejeita); vídeo inexistente / sem API key / sem ffmpeg (preflight). **Combinações válidas úteis:** cache + --top diferente; --force-rescore + --context novo; --min-score 0 p/ smoke test (candidatos failed continuam excluídos).

9. Exemplos de execução

```
python clipper.py video.mkv
python clipper.py video.mkv --out cortes/ --top 8
python clipper.py live.mp4 --cache-dir .cache/clipper --out cortes/
python clipper.py live.mp4 --cache-dir .cache/clipper --out cortes2/ --top 5
python clipper.py live.mp4 --cache-dir .cache/clipper --force-retranscribe
python clipper.py live.mp4 --context "ranked Valorant, duo com fulano"; --examples
examples.json --min-score 6.5
python clipper.py live.mp4 --pad 1.2 --no-audio-features --min-score 5.0
CLIPPER_TRANSCRIBE_BACKEND=cpu python clipper.py live.mp4
python clipper.py live.mp4 --transcribe-backend opencv --model nvidia/nemotron-3-super-120b-
a12b
```

Todos derivam de flags reais (§8); os de cache/context constam no README e no código.

10. Fluxo completo do sistema

Entrada → processamento → saída	Motor (CPU/GPU/remoto)	Erros possíveis
caminho → checagens + sondas → aborta ou segue	CPU	arquivo ausente, sem áudio, s 1 GB
caminho → stat + ffprobe → dict (tamanho, duração, codec...)	CPU	tudo nullable; nunca carrega o
vídeo → ffmpeg chunks 30 s (GPU) ou decode único (CPU)	CPU (ffmpeg)	timeout; chunk vazio encerra e
áudio → Whisper → segmentos+palavras	GPU→CPU (fallback explícito)	backend indisponível; CPU fall
segmentos → janelas 20–90 s → snap ±1.5 s	CPU puro	0 janelas = exit 'sem fala?'
janelas → volumedetect → alta/media/baixa	CPU (1 ffmpeg/cand, 8 s)	timeout → 'media'
lotes de 6 → LLM → score/título/tags	API remota (texto, ≤1500 chars/cand)	lote failed (retry 3×); continua
pontuados → NMS+diversidade → top-N	CPU puro	0 elegíveis = exit
3 frames/clipe → centro x (clamp .25–.75)	CPU (Haar)	sempre há fallback (centro)
dimensões + rosto → scale/crop/pad/subtitles	CPU (filtros)	largura insuficiente → pad
filtros → h264_qsv ■ libx264 → .mp4	GPU→CPU	timeout 600 s; por clipe, sem a
clipes → manifest.json + cache	disco	pasta não gravável (preflight)
tudo → JSON próprio + resumo terminal	CPU (1 thread/2 s)	nunca levanta (best-effort)

11. Arquitetura

Orquestrador fino (clipper.py: CLI + ordem + relatório) sobre core/* com SRP. Dados fluem como objetos (Word/Segment/Candidate); I/O e rede isolados nos módulos donos; sem concorrência entre etapas (um vídeo por processo). Extensão: prompt via --context/--examples sem editar código; novo backend = função em backends.py + ramo em intel.resolve_transcribe_backend.

Responsabilidade → entradas → saídas → dependências → quem usa

orquestra, trata falha/interrupção → argv → mp4+JSONs → todos core/* → entrypoint

entidades + overlap → — → Word/Segment/Candidate → só config → todos

constantes+env+limites → env → valores → system → todo o projeto

specs host + limites → /proc,psutil → dict → — → config, preflight

GPU/QSV/backend → sysfs,lspci,OV,ffmpeg → dict/encoder → openvino → preflight,transcribe,video

fail-fast → caminho+env → exit|None → config,system,intel → clipper

orquestra backends → vídeo → segmentos → cache,intel,backends → clipper

chunks+inferência GPU → vídeo → segmentos → ffmpeg,OV/cpp → transcribe

janelas+snap → segmentos → candidatos → models → clipper

energia → vídeo+janelas → energy/speech_rate → ffmpeg → clipper

LLM em lotes → candidatos → scores → openai,cache → clipper

NMS+diversidade → pontuados → top-N → config → clipper

rosto+SRT+ffmpeg → vídeo+clipe → .mp4 → cv2,intel → clipper

fingerprint+JSON → vídeo/candidatos → hit|None → models → transcribe,scoring

sampler+JSON+resumo → execução → metrics/*.json → psutil → clipper

12. Estrutura de diretórios

```
clipper/
■■■ clipper.py          # CLI + orquestrador (entrypoint)
■■■ core/               # 15 módulos (ver §11; __init__ sem side-effects)
■■■ tools/smoke_gpu.py  # smoke test da B580 (sintético, sem vídeos reais)
■■■ cortes/            # saída padrão (--out); ignorada no git
■■■ metrics/           # 1 JSON por execução (ignorado no git; criado se ausente)
■■■ videos/            # vídeos do usuário (ignorado no git; 17 GB não versionados)
■■■ .cache/            # cache opcional --cache-dir (ignorado no git)
■■■ docs/              # este PDF (versionado)
■■■ examples.json      # few-shot de exemplo p/ --examples (versionado)
■■■ requirements.txt    # deps Python (versionado)
■■■ README.md          # guia de uso (versionado)
■■■ AUDIT-clipper.pdf   # relatório da auditoria pré-execução (versionado)
■■■ .env               # NVIDIA_API_KEY (NUNCA commitado)
■■■ .ai-jail           # config de sandbox da ferramenta (não versionado)
```

Não existe diretório *tests/* nem suite automatizada (ver §26).

13. Transcrição

Aspecto	Implementação real
Modelo	Whisper (WHISPER_MODEL_SIZE; medium padrão, small se RAM ≤ 6 GB; tiny-large via env)
Backends (ordem auto)	vulkan (whisper.cpp) → openvino (GPU) → cpu (faster-whisper); explícito indisponível = CPU + motivo
Devices	GPU p/ vulkan/openvino; CLIPPER_DEVICE (cpu) + int8 p/ faster-whisper; 4 threads, 1 worker
Idioma	CLIPPER_LANGUAGE ou auto; faster-whisper com word_timestamps + VAD
Áudio (GPU)	ffmpeg → WAV 16 kHz mono em chunks de 30 s (~1 MB, 1 vivo por vez, tmp removido)
Áudio (CPU)	decode único do arquivo (~0,4 GB p/ 100 min) — aceitável, auditado
Temporários	\$TMPDIR/clipper_audio_*/chunk_*.wav (+ .json do whisper.cpp); finally remove tudo
Cache	.cache/.../transcript.json (segmentos+palavras); hit pula inferência
B580	CTranslate2 é CUDA-only (nunca usa a B580); por isso OpenVINO/Vulkan dedicados; aqui: OV só vê CPU (sem Level Zero) → CPU explícito
Limites	GPU 1 tentativa cada (sem loop); OOM cai no fallback com motivo; sem realocação

14. GPU e aceleração

Dispositivo real	Detalhe
CPU (sonda)	vendedor 8086 em /sys/class/drm ou lspci; nome do lspci
B580 (se runtime) senão CPU	OpenVINO device GPU ou whisper.cpp/Vulkan; fallback faster-whisper int8
B580 via h264_qsv (se sonda passar)	sonda: encode real 10 frames, 15 s, 1×/processo; senão libx264
CPU	decode software; scale/crop/pad/subtitles + Haar (leves)
API remota	sem GPU local envolvida
CPU	psutil; nvidia-smi só se existir (aqui ausente → null)

VRAM: Whisper-medium ≈ 2–4 GB de 12 GB; 1 instância por vez; chunks liberam entre si; QSV só durante cada encode. Diagnóstico: `vainfo`, `python3 -c "import openvino as ov; print(ov.Core().available_devices)"`, `python tools/smoke_gpu.py` (prova objetiva: `detect/backend/modelo/inferência`), linhas `GPU detected / Transcription backend / Video acceleration` no stdout. Modo `--transcribe-backend gpu` exige GPU e falha claramente se indisponível.

15. Processamento de vídeo

ffmpeg por clipe: `ffmpeg -threads 2 -y -ss <coarse> -i IN -ss <fine> -t <dur> -vf <filtros> -c:v (h264_qsv|libx264) -c:a aac -b:a 160k OUT` (seek duplo = rápido + frame-accurate; timeout 600 s; stdout DEVNULL, stderr em memória — KBs). **Decoder:** software. **Encoder:** h264_qsv (global_quality 23, look_ahead 1) com fallback libx264 (veryfast, crf 20).

9:16: scale=-2:1920 + crop 1080×1920 centrado na mediana do rosto (clamp [0.25,0.75]); se a largura não comporta, scale+pad com barras. Sem --vertical: scale 1280px. **Rosto (OpenCV Haar):** 3 amostras (15/50/85% do clipe), maior face, mediana; sem cv2/cascade/câmera → 0.5. **Legenda:** SRT das palavras (pausa >0,4 s, frase, 8 palavras ou 64 chars → nova cue; mín. 1,0 s; maiúsculas; Arial Black 16, contorno 3). **Intermediários:** só .srt temporário por clipe (KBs, unlink em finally). Saída: `NN_<titulo_sanitizado_≤60>.mp4`.

16. NVIDIA API

Aspecto	Valor real
Endpoint	<code>https://integrate.api.nvidia.com/v1</code> (chat.completions, OpenAI SDK)
Auth	header Bearer NVIDIA_API_KEY (env; checada no preflight e no scoring)
Modelo	--model / NIM_MODEL; default z-ai/glm-5.3 (decisão original GLM; vivo em 2026-09-18)
Payload	system(prompt+contexto+few-shot) + user({clips:[{id, transcript≤1500, start,end,duration,energy,speech_rate}]})
Resposta	{clips:[{id, score 0–10 clamp, reason≤200, title≤50, hashtags=3}]; reasoning (<think>/``) removido
Nº requests	ceil(candidatos/6) por execução (~50 p/ 100 min) + retries
Retry/timeout	3× backoff 10/30/60 s; client timeout=180 s, max_retries=0; lote falho = failed, segue
Rate limit/erros	HTTP/JSON/rede → log + marca failed; métricas contam tudo
Tokens/custo	somente se a API retornar usage; custo sempre null
Privacidade	VÍDEO E ÁUDIO NUNCA SÃO ENVIADOS — só trechos de texto

17. Métricas

Coleta: sampler (1 thread daemon, 1 amostra/2 s, só escalares) + hooks de tempo/contadores nas etapas + stat/ffprobe do vídeo. Impacto desprezível (medido: sem carga perceptível). Falhas de métrica nunca quebram o job (best-effort). Arquivo: `metrics/AAAA-MM-DD_HHMMSS_<stem≤40>_<uid6>.json` (timestamp+uid: reexecuções nunca colidem/sobrescrevem).

17.1 Referência de campos

	Tipo	Significado
	int	versão do schema (1)
size_human	str/int	identificação e tamanho via stat (null se ausente)
hms	float/str	duração via ffprobe (null sem ffprobe)
/codec/fps	str/int/float	stream v:0 via ffprobe (null se indisponível)
_at	ISO-8601	início/fim com timezone
tion_hms	float/str	tempo total de parede
	success failed interrupted	resultado (Ctrl+C = interrupted)
	str null	etapa do abort (preflight...cutting)
ge}	obj null	exceção/SysExit (msg ≤500 chars)
	int	soma de retries (transcrição+API)
	obj	segundos por etapa
	obj	flags/valores efetivos (top, model, threads...)
d_requested,backend_used}	str	modelo Whisper; backend pedido vs efetivo ('cache' em hit)
	str	device efetivo; nome da GPU detectada
hms,rtf}	num	tempo e RTF = tempo/duração (menor = mais rápido)
ack,tried_backends,error}	mix	nº segmentos; se houve fallback; ordem tentada; erro
ices,backend}	mix	GPU lspci; devices OV; backend usado
ct,vram_peak_mb}	num null	nvidia-smi se existir (aqui sempre null — honesto)
_pct,peak_pct,process_cpu_sec}	mix	% relativos a 1 core (400% = 4 cores) + CPU total
_mb,available_mb}	num	RSS do processo + disponível do sistema
ncoder}	str null	qsv software; h264_qsv libx264; decoder sempre null (software, sem sonda)
ips_failed,success}	mix	tempo total de corte + placar
successes,failures,retries}	mix	placar real de chamadas
vg_latency_sec}	num	tempo e latência média por tentativa
on,total}_tokens	int null	só se a API retornar usage
	null	sempre null (sem tabela de preços)
d,clips_ok,clips_failed}	int null	funil resumido

18. Cache

Local: `--cache-dir` (ex. `.cache/clipper`; desligado por padrão). Conteúdo: `<fp>.transcript.json` ({fingerprint, segments[{text,start,end,words[]}]} e `<fp>.scores.json` ({fingerprint, scores[{score,reason,title,hashtags,failed,energy,speech_rate}]}). Fingerprint = `sha1(path|size|mtime|whisper_model|lang)[:16]` — **trocar o arquivo invalida sozinho** (size/mtime mudam). Uso: hit pula transcrição/scoring (scores exigem mesmo nº de candidatos). Limpar: `rm -rf .cache/clipper` ou `--force-retranscribe/--force-rescore`. Risco: arquivo alterado sem mudar size+mtime (raro) reutiliza cache — use `--force-*` nesse caso.

19. Saídas

	Formato / conteúdo	Finalidade
	H.264 + AAC, 1080×1920 (ou 1280px), legenda queimada	clipes postáveis
	lista {file,start,end,original_*,snapped,score,energy,speech_rate,title,hashtags,reason}	índice auditável dos clipes
	schema §17.1 (poucos KB)	fotografia da execução p/ benchmark
	segmentos + palavras	evitar retranscrever
	notas por candidato	evitar repontuar
	logs de progresso + resumo EXECUTION METRICS	acompanhamento e diagnóstico
chunk.wav	~1 MB transitório, removido	alimentar backends GPU
	KBs, removido em finally	queimar legenda

20. Recursos e limites

Limite / comportamento	Vídeo de ~17 GB / 1h40
sem teto imposto (RLIMIT_AS removido, §29-H1); proteção por <code>int8+threads+chunks</code>	estimado poucos GB (modelo ~1 GB + áudio ~0,4 GB + ffmpeg transitório)
modelo por vez; ~2–4 GB de 12 GB	margem folgada; OOM = fallback controlado
whisper 4 th, ffmpeg 2 th, OMP/MKL/OB 4	transcrição CPU leva dezenas de min (RTF ~0,6–0,9); seguro
RAM < 1 GB, aviso < 5 GB; saídas ≈ dezenas de MB	sem risco com 707 GB livres
1 vídeo/processo; 1 subprocesso por vez; etapas sequenciais	sem contenção interna
CPU 3×+180 s; GPU 1×; ffmpeg ≤600 s	job sempre termina ou falha com relatório

21. Segurança e estabilidade

- RAM: nada carrega o vídeo; áudio em chunks (GPU) ou decode único auditado (CPU); sem acúmulo (texto O(segmentos), linear e pequeno).
- VRAM OOM: vira RuntimeError → fallback explícito ou falha controlada; sem loop de realocação; sem múltiplas instâncias.
- CPU: todos os pools limitados (§20); rosto/filtros leves na CPU por desenho.
- Disco/temporários: preflight de espaço; tmpls de MBs com remoção em finally; nenhuma cópia do vídeo.

- Subprocessos: sem shell=True; todos com timeout; run() aguarda e mata em timeout; Ctrl+C de terminal atinge o grupo (filhos morrem).
- Ctrl+C: relatório interrupted + resumo + re-raise; thread de métricas é daemon.
- Originais: guarda saída==entrada; -y só sobre saídas próprias; falha preserva tudo e gera relatório.

22. Tratamento de erros

	Causa provável	Comportamento	Solução
	caminho errado	exit 1 + relatório preflight	conferir caminho
	pacote ausente	exit 1 + relatório	dnf install ffmpeg
da	env ausente	exit 1 + relatório	exportar chave
	driver/modelo/binário	fallback CPU explícito, segue	ler motivo (§23)
t?)	sem fala/VAD	exit + relatório candidates	checar áudio do vídeo
te	scores < min-score	exit + relatório selection	--min-score menor
	modelo expirado	lote failed após 3×; pode zerar elegíveis	--model vivo
264	driver QSV parcial	segue em software, 1 aviso/clipe	drivers; ou aceite CPU
	ffmpeg (filtros/encode)	pula clipe, segue job	log + JSON ffmpeg
	disco < 1 GB	exit 1 + relatório	liberar disco / outro --out

23. Troubleshooting

GPU não detectada

Ver: *GPU detected*: no stdout; *lspci -nn | grep 8086*; *ls /sys/class/drm/*/device/vendor* (0x8086). Se ausente, é hardware/VM — CPU assume.

OpenVINO falhou

Motivos típicos impressos: pacote ausente / sem device GPU (falta o pacote *intel-level-zero*) / optimum ausente / falha de export. Checar: *python3 -c "import openvino as ov; print(ov.Core().available_devices)"* (quer ['CPU','GPU']) + *vainfo* + *python tools/smoke_gpu.py*.

Vulkan falhou

Binário *whisper-cpp/whisper-cli* no PATH? Modelo *models/ggml-<size>.bin* existe? Mensagem diz exatamente o que falta.

Caiu para CPU — por quê?

A linha *Transcription backend*: *CPU* (<motivo>) e *metrics.transcription.{tried_backends,fallback}* dizem o porquê. Comportamento esperado sem drivers.

FFmpeg falhou

Ver o trecho de stderr no log (! QSV falhou... ou ! Falha clipe N: ...); *ffmpeg -version | grep qsv*; testar o comando com -loglevel warning; checar espaço e escrita em --out.

NVIDIA API falhou

Ver `metrics.nvidia_api.{requests,failures,retries,avg_latency_sec}`; 401 = key; 410 = modelo expirado (listar `/v1/models`, trocar `--model`); 429 = rate limit (backoff cobre); timeouts após 180 s/tentativa.

Falta de RAM

Ver `ram.peak_mb` no JSON; rode com `WHISPER_MODEL_SIZE=small`, `--no-audio-features`, feche apps; 17 GB de vídeo ≠ 17 GB de RAM (só metadados).

Falta de VRAM

OOM cai no fallback CPU com motivo. Reduza `WHISPER_MODEL_SIZE` ou force `--transcribe-backend cpu`.

Falta de disco

Preflight já barra (< 1 GB). Checar `df -h` em `--out`, `$TMPDIR` e `~/.cache`.

Processo interrompido

Relatório *interrupted* em `metrics/` diz a etapa. Temporários se limpam sozinhos (`chunks/SRT`); conferir `pgrep -x ffmpeg` (deve estar vazio) e remover clipe parcial se houver.

24. Benchmark

Sem modo especial (YAGNI): **cada execução normal já gera seu próprio relatório**. Para comparar, processe N vídeos (ou o mesmo com backends/modelos diferentes) e agregue os JSONs — exemplo real:

```
python3 -c &quot;
import json, glob
for f in sorted(glob.glob(&#x27;metrics/*.json&#x27;)):
    d = json.load(open(f))
    t, v = d[&#x27;transcription&#x27;], d[&#x27;video&#x27;]
    print(f&quot;{v[&#x27;name&#x27;]} | {v[&#x27;duration_hms&#x27;]} | backend={t[&#x27;
backend_used&#x27;]} | &quot;
        f&quot;RTF={t[&#x27;rtf&#x27;]} | RAM={d[&#x27;ram&#x27;][&#x27;peak_mb&#x27;]}MB |
api={d[&#x27;nvidia_api&#x27;][&#x27;requests&#x27;]}&quot;)
&quot;
```

Compare RTF, tempo por etapa (`execution.stages_sec`), RAM peak, backend e args — nunca conclua só pelo tamanho do arquivo (codec/resolução/duração pesam). Futuro opcional: comando agregador sobre `metrics/` (não implementado).

25. Desenvolvimento

- Estrutura: `clipper.py` fino + `core/*` por responsabilidade (ver §11); sem framework, sem build (script direto).
- Ambiente: mesmo da §6; edite, rode contra vídeo sintético em `/tmp` (nunca os reais sem aval).
- Testes/lint/formatação: não há suite, linter nem formatter configurados — verificação é `python3 -m py_compile` + smoke manual (ver §26).
- Debug: logs de progresso por etapa + metrics JSON (`stages_sec`, `tried_backends`, erros com tipo/mensagem).

- Commits: convenção *flag(scope): mensagem*, inglês, imperativo, conciso (ex. fix(scoring): bound API calls with explicit timeout); correções separadas por área.

26. Testes

Não há testes automatizados no repositório. Procedimento de smoke usado na auditoria (reproduzível, sem vídeos reais):

```
python clipper.py --help # CLI abre
python clipper.py /tmp/inexistente.mp4 # preflight falha COM relatório
ffmpeg -f lavfi -i testsrc=duration=5 -f lavfi -i sine=duration=5 -shortest /tmp/s.mp4
WHISPER_MODEL_SIZE=tiny python clipper.py /tmp/s.mp4 --top 1 --min-score 0 # pipeline real
# Ctrl+C durante o run → metrics/*.json com status interrupted; pgrep -x ffmpeg vazio
```

	O que cobre	Resultado na auditoria
	argparse, detecção B580, relatório de erro	OK (backend CPU + motivo; QSV ok)
	transcribe→scoring→cut→métricas (voz espeak + small)	SUCCESS, RTF 0.86, RAM 0.9 GB
	sem fala, arquivo ausente, modelo expirado	relatórios failed corretos
	SIGINT no ffmpeg + no grupo	interrupted; sem órfãos
	tools/smoke_gpu.py (áudio sintético, sem vídeos)	GPU YES / backend unavailable (falta int
	cpu explícito, auto, gpu exigente (sintético)	CPU SUCCESS; auto→CPU+motivo; gp

27. Dependências

	Versão	Finalidade	Obrigatória
	≥1.0.0 (testada 1.2.1)	Whisper local em CPU via CTranslate2	Sim
	≥1.30.0 (testada 3.15.0)	HTTP do scoring na API NVIDIA	Sim
	≥4.9.0 (testada 5.0.0.93)	Haar Cascade p/ centralizar rosto	Sim
	≥2024.0, só linux (testada 2026.4.0)	runtime + devices da B580	Só backend openvino
soundfile	—	Whisper→OpenVINO + leitura de WAV	Só backend openvino
model	—	transcrição Vulkan na B580	Só backend vulkan
	testada 7.2.2	CPU/RAM em metrics.py e system.py	Sim
	sistema (testado 8.1.2)	áudio, corte, encode, sondas	Sim
	testada 5.0.1	gerar esta documentação	Não (docs)

28. Limitações conhecidas

- Scoring é textual + energia: jogada visual silenciosa sem visão computacional continua difícil.
- Haar Cascade simples (webcam fixa); fora de cena → centro. Legenda cobre parte inferior do frame.
- Modelo padrão anterior expirou (410); default atual verificado vivo em 2026-09-18 — conferir periodicamente.
- Áudio adiciona ~0,5–1 s/candidato (desligável). faster-whisper nunca usa B580 (CUDA-only).

- Sem Level Zero/OpenCL, OpenVINO vê só CPU (estado atual desta máquina). Filtros/decoder sempre CPU por desenho.
- Word-timestamps aproximados nos backends GPU (`_even_words`). `whisper.cpp` recarrega o modelo por chunk (lento).
- Cache por tamanho+mtime: edição que preserve ambos reutiliza cache (use `--force-*`). Sem suite automatizada.

29. Auditoria de estabilidade

Auditoria pré-execução de 2026-09-18 (relatório integral em *AUDIT-clipper.pdf*): leitura total do código + grep de padrões perigosos + testes só com vídeos sintéticos; nenhum vídeo real executado.

```
CRITICAL: 0
HIGH: 4 (todos corrigidos antes dos testes)
MEDIUM: 6 (1 com hardening, 5 aceitos/documentados)
LOW: 3 (documentados)
```

HIGH corrigidos

	Correção
6 GB: teto virtual (não RSS), herdado pelo ffmpeg	removido de <code>core/transcribe.py</code>
Timeout (default 600 s) + retries internos do SDK	<code>timeout=180, max_retries=0</code> em <code>core/scoring.py</code>
Relatório	handler → interrupted em <code>clipper.py</code> (testado)
Deve coincidir com a entrada	guarda <code>resolve()==resolve()</code> em <code>clipper.py</code>

MEDIUM / LOW (resumo)

M1 decode integral CPU ~0,4 GB/100 min (aceito); M2 reload ggml por chunk (aceito); M3 sem checagem de disco → **hardening aplicado** (falha <1 GB/aviso <5 GB); M4 OV export+download no 1º uso; M5 timestamps uniformes em GPU; M6 default expirado (corrigido: decisão original GLM, z-ai/glm-5.3, vivo em 2026-09-18). L1–L3: env de threads, filtros em CPU por desenho, `which()` por amostra.

30. Changelog / estado atual

Sem versões numeradas; histórico real do git (5 commits):

```
6fa5571 fix(scoring): restore original GLM model as default
21e8c1c docs(project): refresh reference for model and gpu mode
0eec2aa docs(readme): update model, backends and smoke test
914b646 fix(gpu): add explicit gpu mode and smoke test
e1dcdb1 fix(deps): declare psutil requirement
7998406 fix(nim): update default model to live slug
... (auditoria fix/* + feat(metrics) abaixo)
```

Estado: pipeline completo + métricas por execução + hardening da auditoria; pendente apenas o primeiro teste em vídeo real (começar pelo menor).

31. Referência rápida

INSTALAÇÃO

```
sudo dnf install -y ffmpeg intel-level-zero intel-media-driver
pip install -r requirements.txt [openvino optimum-intel transformers soundfile]
export NVIDIA_API_KEY=nvapi-... [NIM_MODEL=...]
```

EXECUÇÃO

```
python clipper.py video.mkv [--out cortes/] [--top 8] [--model ...]
[--transcribe-backend auto|gpu|vulkan|openvino|cpu] [--cache-dir .cache/clipper]
[--pad 0.8] [--min-score 6.0] [--max-per-10min 2] [--context T] [--examples J]
[--no-vertical] [--no-captions] [--no-audio-features] [--force-retranscribe] [--force-
rescore]
VARS: NVIDIA_API_KEY* NIM_MODEL CLIPPER_TRANSCRIBE_BACKEND WHISPER_MODEL_SIZE
CLIPPER_LANGUAGE CLIPPER_CPU_THREADS CLIPPER_FFMPEG_THREADS CLIPPER_COMPUTE_TYPE
CLIPPER_DEVICE [CLIPPER_RAM_LIMIT_GB informativo]
BACKENDS: auto → vulkan → openvino(GPU) → cpu (sempre com motivo); gpu = exige GPU, falha
claro
SAÍDAS: cortes/NN_*.mp4 + manifest.json | metrics/<ts>_<video>_<id>.json | .
cache/
DIAG: GPU detected:/backend:/Video acceleration: no stdout; ram/transcription/nvidia_api no
JSON
```